SZABIST
University

Shaheed Zulfikar Ali Bhutto
Institute of Science and Technology

ROBOTICS & ARTIFICIAL INTELLIGENCE DEPARTMENT

Total Marks:_____

Obtained Marks: _____

Computing Vision

Lab Task # 06

Submitted To: **Mr. Muhammad Azhar**

Student Name: **Ayesha Munir**

Reg. Number: **23108106**

Code Example 1: Sobel Edge Detection

In autonomous driving systems, detecting road lanes and edges is crucial for vehicle navigation and safety. Sobel edge detection helps highlight vertical and horizontal edges in grayscale camera images, making lane boundaries, road edges, and obstacles more distinguishable.

By extracting these edges, the system can better interpret the driving environment, plan paths, and avoid hazards.

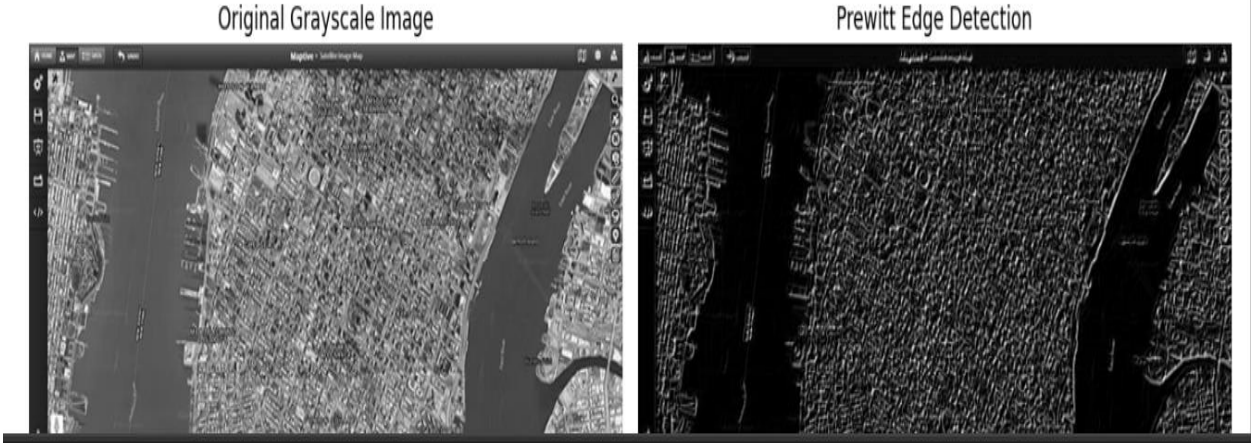
```
]: # Code Example 1: Sobel Edge Detection
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/A18.jpg", cv2.IMREAD_GRAYSCALE)
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel_magnitude = cv2.magnitude(sobelx, sobely)
sobel_norm = np.uint8(255 * sobel_magnitude / np.max(sobel_magnitude))
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(sobel_norm, cmap='gray')
plt.title("Sobel Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Code Example 2: Prewitt Edge Detection

In urban planning and geographic information systems (GIS), detecting the edges of buildings and infrastructure from aerial or satellite images is vital for mapping and analysis. The Prewitt operator, a simple edge detector, highlights horizontal and vertical edges efficiently, making it useful for identifying rectangular building shapes and boundaries.

This helps in automated mapping, urban growth analysis, and disaster management.

```
# Code Example 2: Prewitt Edge Detection
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/A7.jpg", cv2.IMREAD_GRAYSCALE)
kernelx = np.array([[1, 0, -1],
                    [1, 0, -1],
                    [1, 0, -1]],
                   dtype=np.float32)
kernely = np.array([[1, 1, 1],
                    [0, 0, 0],
                    [-1, -1, -1]],
                   dtype=np.float32)
img_prewittx = cv2.filter2D(img, -1, kernelx)
img_prewitty = cv2.filter2D(img, -1, kernely)
prewitt = cv2.addWeighted(img_prewittx, 0.5, img_prewitty, 0.5, 0)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(prewitt, cmap='gray')
plt.title("Prewitt Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```

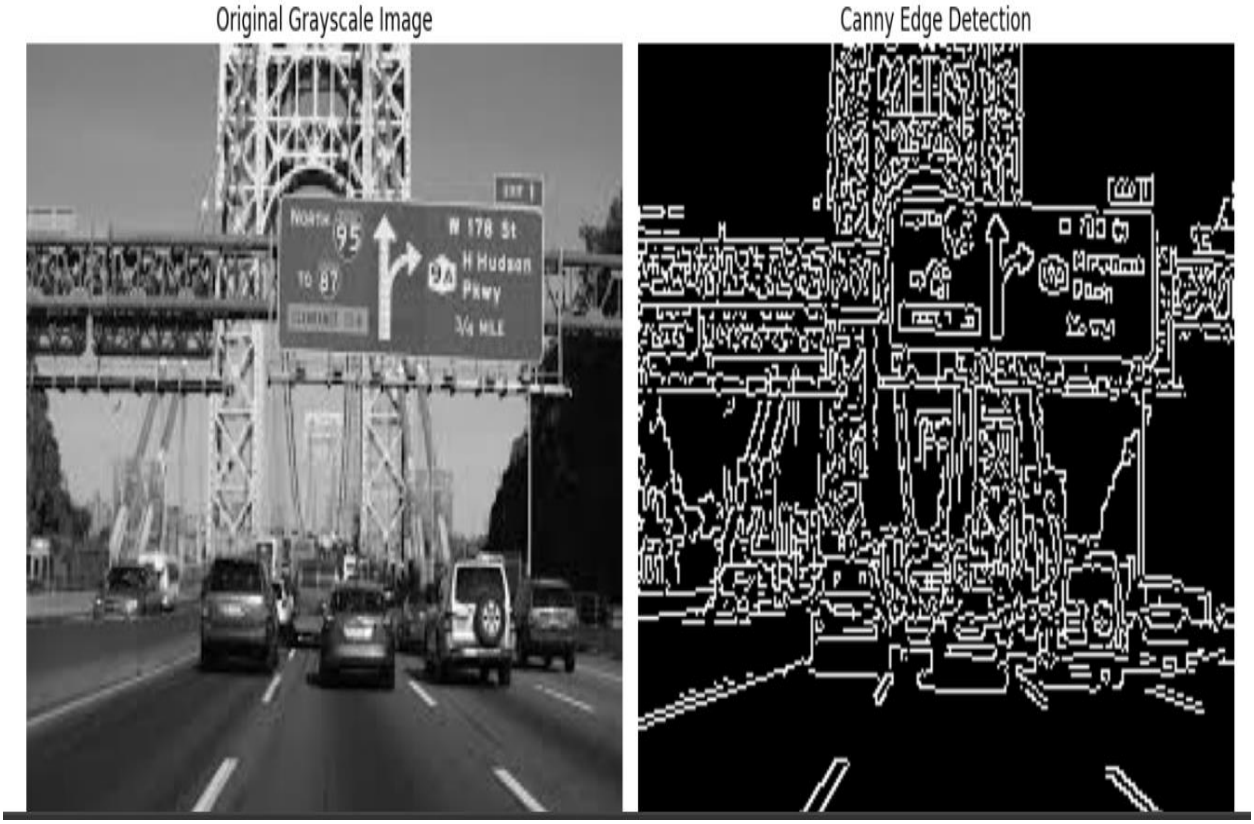


Code Example 3: Canny Edge Detection

Autonomous vehicles rely on detecting and recognizing traffic signs accurately to navigate roads safely. Traffic signs often have clear, sharp edges that define their shape (e.g., stop signs, speed limits).

Using the Canny edge detection algorithm helps extract these sharp edges efficiently, enabling the system to isolate signs from complex backgrounds for further classification and decision making.

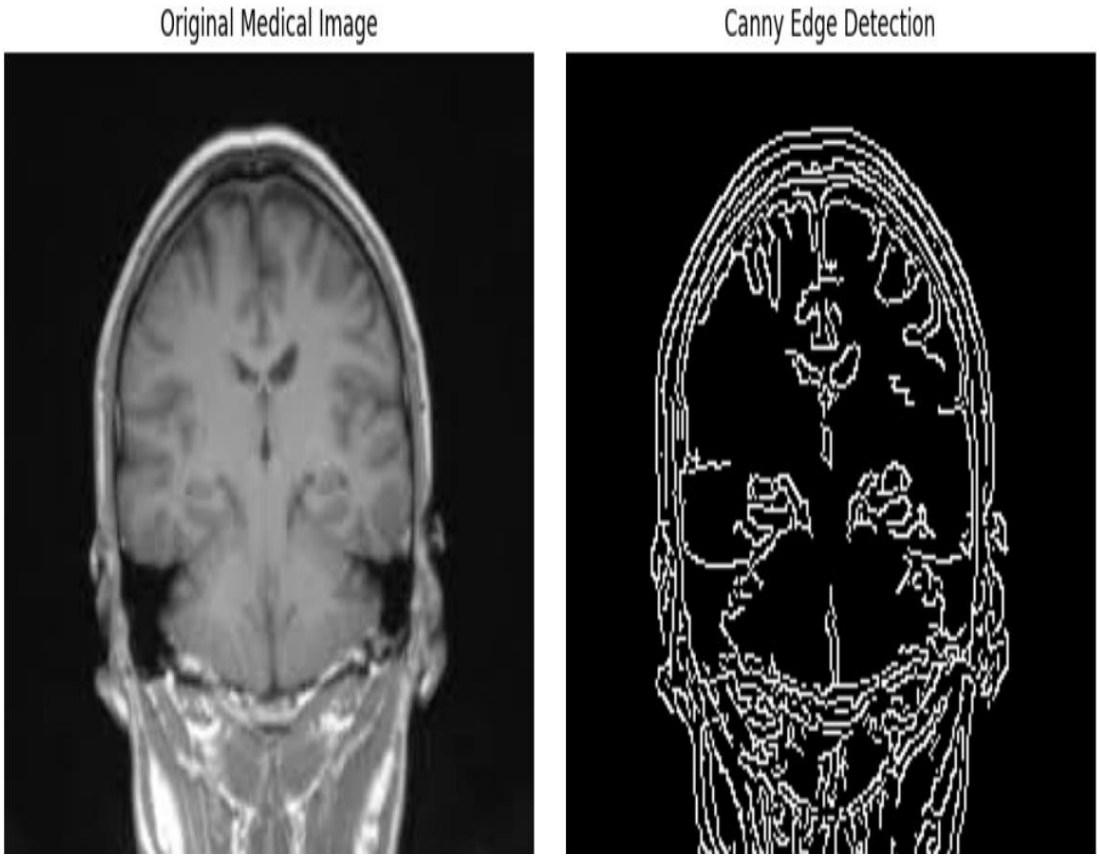
```
# Code Example 3: Canny Edge Detection
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/A19.jpg", cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(img, threshold1=100, threshold2=200)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Case Study 1: Medical Image Edge Detection (CT/MRI Scans)

In medical imaging, radiologists use CT or MRI scans to analyze internal organs and tissues. Edge detection is used to highlight boundaries of tumors, segment organs, or detect anomalies. Canny edge detection is particularly useful due to its ability to suppress noise while retaining clear, thin edges.

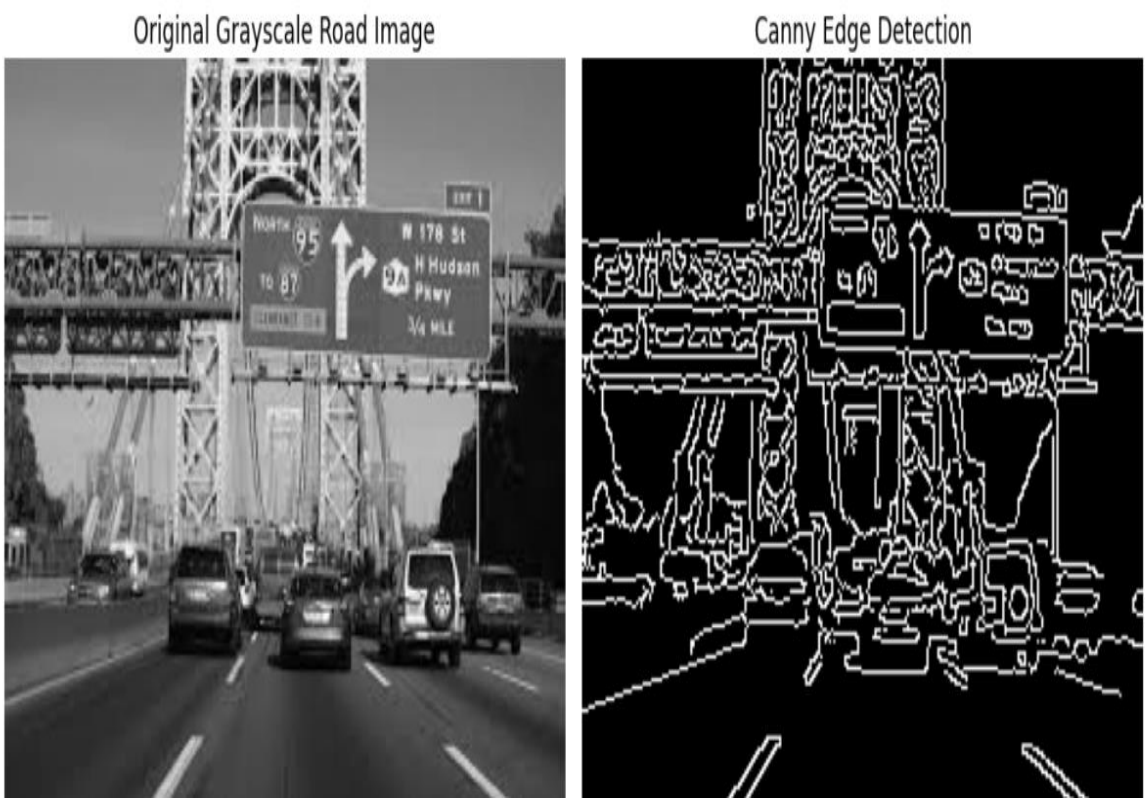
```
# Case Study 1: Medical Image Edge Detection (CT/MRI Scans)
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/A20.jpg", cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(img, 50, 150)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Medical Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Case Study 2: Road and Lane Detection in Autonomous Vehicles

Self-driving vehicles use front-facing cameras to identify lane boundaries, road edges, and traffic signs. Canny edge detection acts as a preprocessing step to simplify the image and extract significant geometric features like white lines, which represent lanes.

```
# Case Study 2: Road and Lane Detection in Autonomous Vehicles
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/A19.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blurred, 50, 150)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Grayscale Road Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Case Study 1: Brain Tumor Boundary Detection in MRI Scans
Scenario: In radiology, precise boundaries of tumors are essential for diagnosis and surgical planning. Edge detection (such as the Canny operator) helps segment the tumor from surrounding brain tissue in grayscale MRI scans.
Objective: Use Canny edge detection to extract and visualize tumor boundaries from MRI scans.

```
# Case Study 1: Brain Tumor Boundary Detection in MRI Scans
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/A20.jpg", cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(img, 50, 150)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Medical Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Case Study 2: Lane Line Detection in Autonomous Vehicles
Scenario: Self-driving cars require reliable detection of lane lines in real-time to maintain safe driving. Edge detection is used as a preprocessing step in lane detection systems to extract lines from road images.
Objective: Apply Sobel or Canny edge detection to road images to extract lane boundaries under various conditions such as low-light or fog.

```python
# Case Study 2: Lane Line Detection in Autonomous Vehicles
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/A18.jpg", cv2.IMREAD_GRAYSCALE)
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel_magnitude = cv2.magnitude(sobelx, sobely)
sobel_norm = np.uint8(255 * sobel_magnitude / np.max(sobel_magnitude))
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(sobel_norm, cmap='gray')
plt.title("Sobel Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```

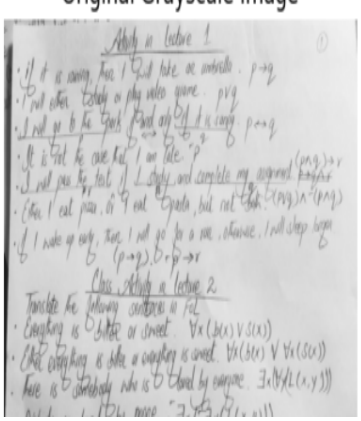
Original Grayscale Image

Sobel Edge Detection

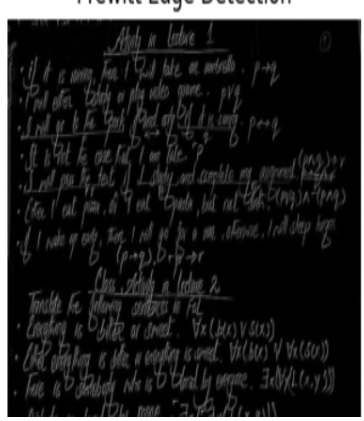
Case Study 3: Signature Extraction from Scanned Documents
Scenario: In document verification, scanned forms often contain noise and printed text. Edge detection helps isolate handwritten signatures by filtering out printed content and enhancing pen strokes.
Objective: Use Prewitt or Laplacian operators to detect signature strokes from scanned grayscale documents.

```python
# Case Study 3: Signature Extraction from Scanned Documents
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/AA5.jpeg", cv2.IMREAD_GRAYSCALE)
kernelx = np.array([[1, 0, -1],
                    [1, 0, -1],
                    [1, 0, -1]],
                   dtype=np.float32)
kernely = np.array([[1, 1, 1],
                    [0, 0, 0],
                    [-1, -1, -1]],
                   dtype=np.float32)
img_prewittx = cv2.filter2D(img, -1, kernelx)
img_prewitty = cv2.filter2D(img, -1, kernely)
prewitt = cv2.addWeighted(img_prewittx, 0.5, img_prewitty, 0.5, 0)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(prewitt, cmap='gray')
plt.title("Prewitt Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Original Grayscale Image

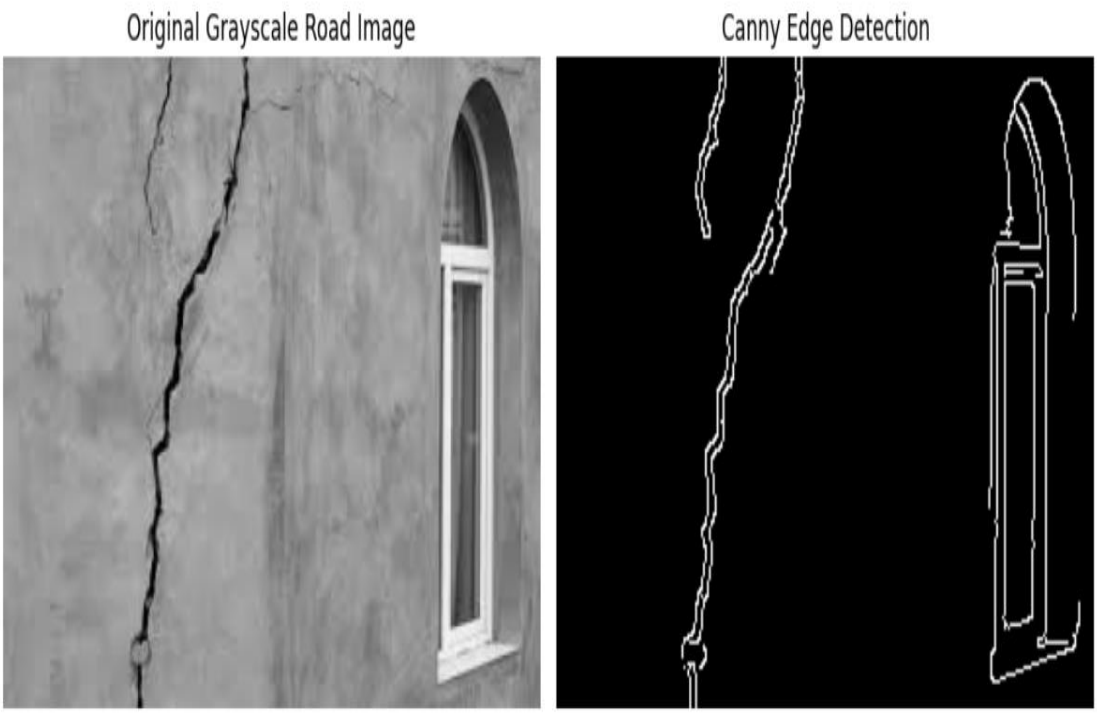


Prewitt Edge Detection



Case Study 4: Structural Crack Detection in Buildings
Scenario: In civil engineering, detecting cracks in structures (such as bridges, dams, and buildings) is crucial for safety assessment. Edge detection can be used to identify fine cracks from high-resolution surface images.
Objective: Apply Canny edge detection to images of building surfaces to highlight potential structural cracks.

```
# Case Study 4: Structural Crack Detection in Buildings
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/A21.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blurred, 50, 150)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Grayscale Road Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Case Study 5: Face Feature Detection for Biometric Systems
Scenario: In facial recognition systems, detecting features such as eyes, nose, and mouth edges is essential for face alignment and identity verification. Edge detection highlights these features before further analysis.
Objective: Use Sobel or Laplacian edge detection to enhance key facial features in grayscale images before recognition.

```
# Case Study 5: Face Feature Detection for Biometric Systems
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108106/A16.jpg", cv2.IMREAD_GRAYSCALE)
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel_magnitude = cv2.magnitude(sobelx, sobely)
sobel_norm = np.uint8(255 * sobel_magnitude / np.max(sobel_magnitude))
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(sobel_norm, cmap='gray')
plt.title("Sobel Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```

https://github.com/Ayesha-Munir-Chohan/